



One AI, one opinion

Single-model answers are fast but unchallenged. You get a confident reply with no adversarial pressure — echo chamber by default.



Multiple agents debate

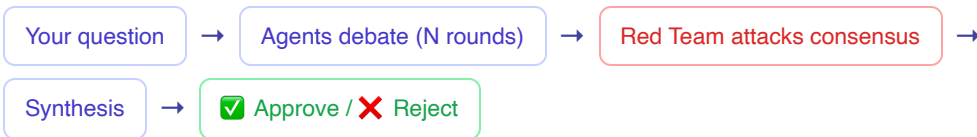
Agent Factory routes your question to 2–5 models that argue both sides. A Red Team agent attacks the dominant view before synthesis.



You decide

The synthesis surfaces a recommended action. You click Approve or Reject. The model that survived adversarial pressure earns more trust.

0 What you'll end up with



Smart routing sends cheap tasks (summarize, commit messages, boilerplate) to free local models and saves API quota for complex reasoning.

1 Choose your interface



Discord Bot ANY DEVICE

Type questions in a Discord channel from your phone, desktop, or tablet. Approve/Reject buttons appear inline. Great if you already use Discord or want mobile access.

Needs: Discord account + a server where you're admin



Local Markdown

**NO ACCOUNT
NEEDED**

Two files: write questions in `inbox.md`, read replies in `conversation.md`. Agent Factory watches for changes. Type `!approve` or `!reject` to act on results.

Needs: Nothing extra — just a text editor

Set `TRANSPORT=both` in your `.env` to run both interfaces simultaneously.

2 Prerequisites checklist

- ☐ **Node.js 18+** — `node --version` · Install at nodejs.org if needed
- ☐ **Discord account + server admin role** — only if using Discord transport
- ☐ **Database** — Docker Desktop (easiest) *or* a free Supabase account (supabase.com)
- ☐ **At least one AI agent** — see section 3 below
- ☐ **Claude Code installed** — for the interactive setup skill (optional but recommended)

3 Choose your agents (pick at least one)

Goal: no per-query API costs. Use subscription CLIs and free local models.



Claude

SUBSCRIPTION

CLI

Requires **Claude Max** subscription (claude.ai). After installing Claude Code, the `claude` binary is already there — zero extra setup.



Gemini CLI

FREE

Google account auth — no API key, no cost. Install: `npm i -g @google/gemini-cli` then `gemini auth` (one-time browser login).



Ollama (local)

LOCAL

Runs on your machine. Install at ollama.ai, then `ollama pull llama3.2`. Needs 8GB+ RAM. Free, private, slower.



Codex CLI

SUBSCRIPTION

Requires OpenAI subscription + Codex desktop app. Mac: binary at [/Applications/Codex.app/Contents/Resources/codex](https://openai.com/app/Contents/Resources/codex)

Note: Configure only agents you have. Unconfigured agents are silently skipped. The bot works with just one.

4A Discord bot setup (~5 min) — skip if using file-only

- 1 **Create an Application**
Go to discord.com/developers/applications → New Application → give it a name → Create
- 2 **Add a bot + enable Message Content Intent**
Left sidebar → **Bot** → Add Bot. Scroll to **Privileged Gateway Intents** → toggle **Message Content Intent** ON → Save Changes. (*Don't skip — without this the bot sees reactions but not message text.*)
- 3 **Copy bot token**
Token section → Reset Token → copy. Save as `DISCORD_BOT_TOKEN` in your `.env`.
- 4 **Invite bot + get channel ID & webhook URL**
OAuth2 → URL Generator → Scopes: `bot` → Permissions: Send Messages, Read Messages, Add Reactions, Embed Links → copy URL → authorize in browser.

Then: Enable Developer Mode (User Settings → Advanced). Right-click channel → Copy Channel ID (`DISCORD_TASK_CHANNEL_ID`). Edit Channel → Integrations → Webhooks → New → Copy URL (`DISCORD_WEBHOOK_URL`).

4B File transport setup — skip if using Discord-only

No Discord account needed. Two files in a folder you choose:

inbox.md

Write your question here and save. Agent Factory clears it immediately (so you know it was picked up) and starts processing.

conversation.md

Read-only append log. Every question + answer is appended here with timestamps. Open in any editor that auto-refreshes.

Add to your `.env` :

```
TRANSPORT=file                                # or "discord" or "both"
CONVERSATION_DIR=~/.agent-chats                # folder for inbox.md + conversation.md
```

5 Database (pick one)

Option A — Docker (local)

Requires Docker Desktop running.

```
cd ai-skills/agent-factory
docker compose up -d
```

DATABASE_URL:

```
postgresql://agent:agentfactory@localhost:5432/agent_factory
```

Option B — Supabase (cloud)

Free tier at
supabase.com. No
Docker needed.

1. New Project → set password
2. Settings → Database → Connection String (URI)
3. Copy the

```
postgresql://...
string
```

4. Run: `psql`

```
"$DB_URL" -f
schema.sql
```

6 Install & configure

```
# Clone the repo
git clone https://github.com/LNS-Research/ai-skills.git
cd ai-skills/agent-factory
npm install
```

Copy the env template and fill it in

```
cp .env.example .env
nano .env # or open in any editor
```

Minimum `.env` for a Claude + Gemini + file transport setup:

```
DATABASE_URL=postgresql://agent:agentfactory@localhost:5432/agent_factory
TRANSPORT=file # "discord", "file", or "both"
CONVERSATION_DIR=~/agent-chats # only needed for file transport

# Discord (only if TRANSPORT=discord or both)
DISCORD_BOT_TOKEN=your_token_here
DISCORD_TASK_CHANNEL_ID=your_channel_id
DISCORD_WEBHOOK_URL=https://discord.com/api/webhooks/...

CLAUDE_BIN=/usr/local/bin/claude # which claude
```

```
GEMINI_BIN=/opt/homebrew/bin/gemini      # which gemini
AGENT_ROUTER_ENABLED=1
```

7 Start & test

```
npm start
# Discord: [discord-bot] logged in as YourBotName#1234
# File:    [file-bot] watching ~/agent-chats/inbox.md
```

Discord test: Type `What is the capital of France?` in your channel. Bot reacts ⚡, replies in ~30s.

File test: Write `What is the capital of France?` in `inbox.md` and save. Inbox clears, answer appears in `conversation.md` in ~30s.

Then try a debate:

```
# Discord channel or inbox.md:
!debate: Should I use Postgres or SQLite for a small side project?
```

After ~2 minutes: Discord shows Approve/Reject/Ask buttons. File transport: write `!approve`, `!reject`, or `!ask: follow-up question` in `inbox.md`.

8 Command reference (same in Discord and file transport)

Command	What it does
<code>What is X?</code>	Auto-routed — local model if available, Claude as fallback
<code>!claude: prompt</code>	Force Claude CLI
<code>!gemini: prompt</code>	Force Gemini CLI (free, no API key)
<code>!local: prompt</code>	Force local Ollama model
<code>!codex: prompt</code>	Queue for next Codex session (async)
<code>!debate: topic</code>	2-agent debate with one Red Team agent
<code>!debate claude vs gemini: topic</code>	Explicit agents
<code>!debate claude vs gemini 3: topic</code>	3 cycles each
<code>!debate claude vs gemini --red gemini: topic</code>	Explicitly set Red Team agent
<code>!approve</code>	Execute the proposed action from last debate

Command	What it does
<code>!reject</code>	Dismiss the proposal
<code>!ask: follow-up</code>	Ask agents to elaborate on the synthesis

Smart routing — saves API quota

Task type	Routes to
<code>summarize</code> , <code>commit_msg</code> , <code>boilerplate</code> , <code>explain_fn</code> , <code>seed_data</code>	Local model (free) → Claude as fallback
<code>long_doc</code> , <code>vision</code> , <code>pdf_analysis</code>	Gemini CLI
<code>chat</code> , everything else	Claude CLI

9 Troubleshooting

Symptom	Fix
Bot doesn't react to messages	Check Message Content Intent is enabled in Developer Portal → Bot → Privileged Gateway Intents. #1 mistake.
<code>Used disallowed intents</code> in logs	Same fix — enable Message Content Intent, restart service.
File bot not responding	Verify <code>TRANSPORT=file</code> or <code>both</code> . Check <code>CONVERSATION_DIR</code> exists. Look for <code>[file-bot] watching...</code> in startup logs.
Database connection refused	Docker: <code>docker compose ps</code> — db must show healthy. Supabase: check password in connection string.
Claude auth error	Run <code>claude auth</code> in terminal to re-authenticate.
Gemini auth error	Run <code>gemini auth</code> to re-authenticate with Google account.
Debate times out	Local models (Ollama) are slow. Use <code>!debate claude vs gemini:</code> for fast debates.

Advanced: context injection & auto-start

Add a `CONTEXT_HINTS_JSON` env var so agents automatically receive context about your projects:

```
CONTEXT_HINTS_JSON=[
  {"pattern":"myapp","context":"MyApp is a SaaS tool for ..."},
  {"pattern":"backend","context":"Our backend is Node.js + Postgres on Railway ..."}
]
```

When you type `!debate: should myapp use Redis?`, the context is prepended automatically.

Auto-start on macOS: Run `/setup-agent-factory` in Claude Code and ask it to set up the launchd service — it will write and load the plist automatically.

Linux: Ask Claude: "Set up a systemd service to run npm start in /path/to/agent-factory on boot."

Agent Factory · github.com/LNS-Research/ai-skills · Run `/setup-agent-factory` in Claude Code for agent-factory/ interactive guided setup